

HARTSEER FRONTEND DOCUMENTATION

CAPITULO 1 - DISEÑO DEL FRONTEND

INTRODUCCIÓN

El presente documento tiene como objetivo describir el proceso de diseño frontend desarrollado para Hartseer Analytics, detallando los criterios, decisiones y fundamentos que dieron origen a la arquitectura visual y estructural implementada en la aplicación.

Hartseer Analytics constituye un proyecto integrador orientado al área de Data Analytics cuyo propósito consiste en simular el proceso completo de análisis de un negocio de comercio electrónico. A lo largo de las etapas anteriores se desarrollaron el análisis del negocio, el modelado de la base de datos, la implementación del modelo relacional, la validación analítica mediante consultas SQL y la construcción de los dashboards en Power BI.

La presente etapa representa el paso final del proyecto y tiene como finalidad desarrollar la aplicación web que permitirá consumir y presentar toda la información analítica obtenida durante las etapas anteriores. A diferencia de una implementación basada únicamente en Power BI, el desarrollo de un frontend propio busca ofrecer una experiencia de usuario más consistente, flexible y profesional, simulando el funcionamiento de una aplicación de Business Intelligence utilizada dentro de una organización.

El desarrollo del frontend no persigue demostrar conocimientos avanzados de desarrollo web ni competir con frameworks modernos, sino construir una interfaz limpia, mantenible y escalable utilizando HTML5, CSS3 y JavaScript Vanilla, fortaleciendo al mismo tiempo los fundamentos del desarrollo frontend.

Uno de los principios fundamentales adoptados durante esta etapa consiste en priorizar la simplicidad sobre la complejidad. Cada componente incorporado deberá responder a una necesidad concreta del negocio, evitando funcionalidades innecesarias que incrementen la carga visual o amplíen el alcance originalmente definido para la versión inicial del proyecto.

El presente capítulo documenta el proceso de diseño conceptual que dio origen a la arquitectura del frontend de Hartseer Analytics. Las decisiones descritas en las siguientes secciones constituyen la base sobre la cual posteriormente se desarrolló la implementación completa de la aplicación, manteniendo coherencia

con la filosofía general del proyecto y con los objetivos analíticos definidos durante las etapas anteriores.

ALCANCE DEL PROYECTO

El presente documento describe el diseño frontend correspondiente a la versión 1.0 de Hartseer Analytics.

El objetivo de esta etapa consiste en construir una aplicación web capaz de presentar de forma profesional toda la arquitectura analítica desarrollada durante las etapas anteriores, manteniendo una experiencia de usuario consistente y preparada para futuras ampliaciones.

La versión inicial contempla exclusivamente la implementación de los cuatro dashboards analíticos definidos durante la etapa de diseño:

- Executive Dashboard.
- Products Dashboard.
- Customers Dashboard.
- Marketing Dashboard.

Asimismo, la aplicación incorporará un sistema de navegación persistente, filtros dinámicos, visualizaciones interactivas y una arquitectura de datos desacoplada basada en SQL Views, un proceso de exportación desarrollado en Python y archivos JSON consumidos por la interfaz. Esta solución fue adoptada para la versión 1.0 preservando una arquitectura preparada para incorporar una API REST en futuras versiones.

Debido al alcance establecido para esta primera versión, el proyecto no contempla funcionalidades como autenticación de usuarios, generación de reportes descargables, integración con APIs externas, internacionalización, cambio de tema visual (Light/Dark Mode), notificaciones, exploración avanzada de datos o herramientas administrativas. Dichas características forman parte del roadmap previsto para futuras versiones del sistema.

El alcance definido busca priorizar la construcción de una solución sólida desde el punto de vista arquitectónico y de experiencia de usuario, asegurando que cada componente implementado responda a una necesidad claramente identificada antes de incorporar nuevas funcionalidades.

ARQUITECTURA FRONTEND

El frontend de Hartseer Analytics fue concebido siguiendo una arquitectura basada en una **Single Page Application (SPA)**, donde la navegación entre los distintos dashboards ocurre dentro de un único documento HTML.

Esta decisión permite evitar recargas completas de la aplicación, mantener una experiencia de usuario continua y reutilizar los componentes compartidos por todas las vistas del sistema. De esta manera, la Sidebar, la estructura general del layout y los estilos globales permanecen activos mientras únicamente se reemplaza el contenido principal correspondiente a cada dashboard.

Para la versión 1.0, la arquitectura de datos quedó organizada mediante el siguiente flujo:

MySQL

↓

SQL Views

↓

Python Export

↓

JSON

↓

Frontend

La base de datos MySQL almacena la información comercial utilizada por el proyecto. Sobre esta estructura se implementaron SQL Views destinadas a consolidar los datos necesarios para el análisis, centralizando las relaciones entre tablas y exponiendo una estructura más adecuada para su consumo posterior.

A partir de estas vistas, un proceso de exportación desarrollado en Python extrae la información y genera archivos JSON. Estos archivos constituyen la fuente de datos utilizada por la aplicación frontend durante la versión 1.0.

El frontend consume dicha información mediante una capa de acceso a datos y la transforma posteriormente en indicadores, comparaciones y visualizaciones dinámicas. De esta manera, los dashboards pueden actualizar sus KPI, gráficos y filtros sin acceder directamente a la base de datos ni incorporar lógica SQL dentro de la interfaz.

Esta separación de responsabilidades permite mantener una arquitectura modular y desacoplada, donde cada capa cumple una función específica dentro del flujo de información. Asimismo, la solución conserva la posibilidad de sustituir en futuras versiones la capa de archivos JSON por una API REST, sin necesidad de modificar la estructura principal de la aplicación ni la lógica de presentación desarrollada.

SISTEMA DE NAVEGACIÓN

La aplicación fue concebida como una herramienta de **Business Intelligence** y no como un sitio web tradicional. En consecuencia, el sistema de navegación fue diseñado priorizando la continuidad de la experiencia de usuario, el acceso inmediato a la información y la rapidez de interacción entre los distintos módulos analíticos.

Por este motivo se adoptó un sistema de navegación persistente basado en una **Sidebar** fija ubicada sobre el lateral izquierdo de la interfaz. Este componente constituye el principal mecanismo de navegación de la aplicación y permanece visible durante toda la interacción del usuario, proporcionando acceso permanente a los distintos dashboards sin modificar la estructura general del sistema.

La navegación se organiza en tres grupos funcionales claramente diferenciados:

Analytics

- Executive
- Products
- Customers
- Marketing

Roadmap

- Reports (v1.1)
- Alerts (v1.2)
- Data Explorer (v1.2)

Resources

- Documentation

Los módulos pertenecientes al apartado **Roadmap** permanecen visibles con el objetivo de comunicar la evolución prevista del proyecto y anticipar las funcionalidades contempladas para futuras versiones. Si bien dichos módulos no forman parte del alcance de la versión 1.0, su presencia dentro de la interfaz busca representar una planificación de producto más cercana a la utilizada en aplicaciones empresariales reales.

La navegación entre dashboards se implementa mediante una arquitectura **Single Page Application (SPA)**, donde únicamente se actualiza el contenido principal de la aplicación. Tanto la Sidebar como los componentes compartidos permanecen constantes durante toda la navegación, eliminando recargas completas de la página y proporcionando una experiencia de usuario más fluida y consistente.

Este enfoque permite además reutilizar la totalidad de los componentes compartidos entre dashboards, reducir la duplicación de código y mantener una arquitectura fácilmente escalable para futuras ampliaciones del sistema.

SISTEMA DE COMPONENTES

La interfaz de Hartseer Analytics fue diseñada siguiendo una arquitectura basada en **componentes reutilizables**, donde cada elemento de la aplicación cumple una única responsabilidad y puede ser utilizado por múltiples dashboards sin modificar su comportamiento ni su implementación.

Este enfoque permite mantener una estructura consistente entre todas las vistas del sistema, reducir la duplicación de código y facilitar tanto el mantenimiento como la incorporación de nuevas funcionalidades en futuras versiones. Asimismo, la reutilización de componentes garantiza una experiencia de usuario homogénea, preservando la coherencia visual y funcional de toda la aplicación.

Los componentes definidos durante la etapa de diseño se agrupan de acuerdo con la responsabilidad que cumplen dentro de la interfaz.

Layout

- App Layout
- Sidebar
- Dashboard Header
- Dashboard Content

Los componentes de Layout constituyen la estructura principal de la aplicación y definen la distribución general de cada dashboard, manteniendo una organización uniforme entre todas las vistas.

Navegación

- Navigation Item
- Navigation Section
- Version Badge
- Documentation Link

Estos componentes son responsables de organizar el sistema de navegación persistente implementado mediante la Sidebar, proporcionando acceso a los distintos módulos de la aplicación y comunicando visualmente el estado de las funcionalidades previstas para futuras versiones.

Filtros

- Date Filter
- Dropdown Filter
- Filter Group

Los componentes de filtrado permiten modificar dinámicamente la información presentada en cada dashboard. Su diseño contempla la reutilización de una misma estructura base, adaptándose posteriormente a las necesidades particulares de cada módulo analítico.

Analítica

- KPI Card
- Chart Card

Estos componentes representan el núcleo visual de la aplicación. Las **KPI Cards** muestran los principales indicadores de negocio, mientras que las **Chart Cards** encapsulan las distintas visualizaciones analíticas utilizadas para representar la información de manera gráfica y facilitar su interpretación.

Componentes Base

- Button
- Divider
- Icon

Estos elementos constituyen componentes auxiliares utilizados por el resto de la interfaz para mantener una apariencia consistente y reducir la repetición de elementos comunes.

Estados

- Loading
- Empty State
- Error State

Los componentes de estado fueron definidos para representar las distintas situaciones que pueden producirse durante la interacción del usuario con la aplicación, proporcionando una respuesta visual consistente frente a procesos de carga, ausencia de información o errores inesperados.

La definición de una arquitectura basada en componentes reutilizables constituye uno de los principios fundamentales del frontend de Hartseer Analytics. Esta decisión no solo simplifica el mantenimiento del código y favorece su escalabilidad, sino que además permite incorporar nuevas funcionalidades sin modificar la estructura principal de la aplicación, preservando la consistencia visual y funcional entre todos los dashboards.

SISTEMA VISUAL

El sistema visual de Hartseer Analytics fue diseñado tomando como referencia aplicaciones modernas utilizadas dentro del ámbito empresarial, entre ellas **Linear, Stripe, Vercel, GitHub, Notion y Raycast**, reconocidas por priorizar la claridad de la información, la consistencia visual y una experiencia de usuario orientada a la productividad.

La interfaz fue concebida bajo una filosofía **minimalista**, donde el objetivo principal consiste en facilitar la interpretación de la información analítica antes que destacar mediante efectos visuales o elementos decorativos. Cada componente presente en la aplicación responde a una necesidad funcional específica, evitando incorporar recursos gráficos que no aporten valor al proceso de análisis.

La versión 1.0 fue desarrollada exclusivamente en **modo oscuro**, utilizando una paleta compuesta principalmente por tonos negros, grises, azules y violetas. Esta elección busca mejorar el contraste entre los distintos elementos de la interfaz,

reducir el ruido visual y proporcionar una estética alineada con aplicaciones profesionales de Business Intelligence y análisis de datos.

La jerarquía visual de la aplicación se establece mediante el uso combinado de tipografía, tamaño, espaciado, contraste y alineación de los componentes. De esta manera, la atención del usuario se dirige de forma natural hacia los indicadores de mayor relevancia, favoreciendo una lectura rápida de la información sin necesidad de incorporar elementos visuales excesivos.

Dentro de cada dashboard, las **KPI Cards** ocupan siempre la primera posición de la interfaz, proporcionando una visión resumida del estado del negocio. A continuación, se presentan las visualizaciones analíticas, organizadas de acuerdo con la naturaleza de la información representada. Para ello se emplean principalmente gráficos de líneas, barras y dona, manteniendo criterios de representación consistentes entre los distintos módulos de la aplicación.

Asimismo, se priorizó una distribución basada en amplios espacios entre componentes, bordes suavemente redondeados, sombras discretas y una alineación uniforme de todos los elementos visuales. Estas decisiones contribuyen a mejorar la legibilidad de la información y a proporcionar una experiencia de navegación consistente a lo largo de toda la aplicación.

El sistema visual desarrollado para Hartseer Analytics busca establecer un equilibrio entre estética y funcionalidad, donde el diseño actúa como un soporte para el análisis de la información y no como un elemento protagonista. Esta filosofía permite mantener una interfaz limpia, profesional y preparada para incorporar nuevas funcionalidades sin comprometer la coherencia visual del sistema.

DECISIONES DE DISEÑO

Durante el proceso de diseño del frontend se adoptaron diversas decisiones orientadas a construir una aplicación simple, coherente y preparada para evolucionar de manera progresiva sin comprometer la mantenibilidad del proyecto. Cada una de estas decisiones buscó equilibrar la experiencia de usuario, la claridad visual y la simplicidad de implementación, priorizando aquellos elementos que aportaran valor directo al proceso de análisis.

Una de las primeras decisiones consistió en eliminar la **Topbar** presente en el diseño conceptual inicial. Las funcionalidades previstas para este componente — como búsqueda global, cambio de tema, alertas y acciones generales— excedían

el alcance establecido para la versión 1.0. En consecuencia, se optó por simplificar la estructura de navegación, trasladando la información relevante de cada módulo a un encabezado propio compuesto por un título, un subtítulo descriptivo y los filtros correspondientes.

La navegación fue concebida mediante una **Sidebar persistente**, permaneciendo visible durante toda la interacción del usuario. Esta decisión permite mantener un acceso permanente a los distintos dashboards, eliminar la duplicación de elementos comunes y ofrecer una experiencia similar a la utilizada por aplicaciones modernas de Business Intelligence.

Asimismo, se decidió implementar una arquitectura basada en **Single Page Application (SPA)**, donde únicamente se actualiza el contenido principal de la aplicación. Este enfoque evita recargas completas entre dashboards, mejora la continuidad de la experiencia de usuario y favorece la reutilización de componentes compartidos.

Los filtros fueron diseñados para responder a las necesidades específicas de cada dashboard. Mientras que los módulos **Executive** y **Customers** incorporan exclusivamente filtros temporales, el dashboard **Products** añade un selector para modificar la dimensión de análisis y el dashboard **Marketing** incorpora un filtro específico por canal. Esta decisión permite mantener una interfaz uniforme sin limitar las capacidades analíticas particulares de cada módulo.

Otra decisión relevante consistió en adoptar una arquitectura basada en **componentes reutilizables**, evitando desarrollar implementaciones específicas para cada dashboard. Componentes como las KPI Cards, los filtros, los encabezados y las tarjetas de gráficos fueron concebidos para reutilizarse en toda la aplicación, reduciendo la duplicación de código y simplificando el mantenimiento del sistema.

Finalmente, durante la evolución del proyecto se redefinió la estrategia de acceso a los datos. Si bien el diseño conceptual contemplaba el consumo de información mediante una API REST, para la versión 1.0 se optó por implementar una capa intermedia basada en **SQL Views**, un proceso de exportación desarrollado en **Python** y archivos **JSON** consumidos directamente por el frontend. Esta decisión permitió desacoplar completamente la interfaz del modelo relacional, simplificar el proceso de despliegue y preservar una arquitectura preparada para incorporar una API REST en futuras versiones sin modificar la lógica de presentación implementada.

En conjunto, estas decisiones permitieron construir un frontend modular, consistente y fácilmente escalable, manteniendo el equilibrio entre simplicidad de implementación, experiencia de usuario y capacidad de evolución del proyecto.

ARQUITECTURA DE INTEGRACIÓN

La arquitectura de integración fue concebida con el objetivo de desacoplar completamente la interfaz de usuario de la fuente de datos, garantizando una clara separación de responsabilidades entre las distintas capas que componen la aplicación.

Para la versión 1.0 de Hartseer Analytics, la comunicación entre los distintos componentes del sistema sigue el siguiente flujo:

Usuario

↓

Frontend

↓

JSON

↓

Python Export

↓

SQL Views

↓

MySQL

La información comercial es almacenada en una base de datos MySQL, donde se encuentra normalizada mediante un modelo relacional diseñado específicamente para representar el funcionamiento de un negocio de comercio electrónico.

Sobre esta estructura se implementan **SQL Views**, cuya finalidad consiste en consolidar la información proveniente de múltiples tablas y exponer un conjunto de datos optimizado para el análisis. Estas vistas eliminan la necesidad de reproducir consultas complejas dentro del frontend y centralizan la lógica de obtención de información.

Posteriormente, un proceso de exportación desarrollado en **Python** consulta dichas vistas y genera archivos **JSON**, los cuales constituyen la fuente de datos consumida por la aplicación durante la versión 1.0.

Finalmente, el frontend carga estos archivos mediante una capa de acceso a datos y construye dinámicamente los indicadores, filtros y visualizaciones que conforman cada uno de los dashboards analíticos.

Esta arquitectura permite mantener completamente separadas las responsabilidades de almacenamiento, transformación y presentación de la información. Asimismo, conserva la posibilidad de sustituir la capa de archivos JSON por una API REST en futuras versiones sin modificar la arquitectura general del frontend ni la lógica implementada para el procesamiento de los datos.

ROADMAP

La arquitectura conceptual definida para Hartseer Analytics fue diseñada considerando la evolución progresiva del proyecto, permitiendo incorporar nuevas funcionalidades sin modificar la estructura principal de la aplicación ni comprometer la mantenibilidad del código.

Durante la etapa de diseño se priorizó la construcción de una base sólida y modular, capaz de acompañar el crecimiento del sistema mediante la incorporación incremental de nuevos módulos analíticos, herramientas de gestión y mejoras en la experiencia de usuario.

Las funcionalidades previstas para las próximas versiones son las siguientes:

Versión 1.1

- Reports

Versión 1.2

- Alerts
- Data Explorer

Versiones futuras

- Light Theme
- Internacionalización
- Cambio de moneda
- Integración con APIs externas
- Reportes descargables
- Sistema administrativo
- Gestión logística
- Métodos de pago
- Nuevas métricas analíticas

La incorporación de estas funcionalidades fue contemplada desde las primeras etapas de diseño mediante una arquitectura modular, permitiendo ampliar las capacidades del sistema sin alterar los componentes principales de navegación, presentación y procesamiento de la información.

CONCLUSIÓN

El proceso de diseño desarrollado para el frontend de Hartseer Analytics permitió definir una arquitectura conceptual coherente con los objetivos generales del proyecto, estableciendo los principios que posteriormente guiaron la implementación completa de la aplicación.

La definición de la estructura de navegación, el sistema de componentes, la organización visual y la arquitectura de integración proporcionó un marco de trabajo consistente para el desarrollo del frontend, permitiendo mantener una misma filosofía de diseño a lo largo de todas las etapas de construcción del sistema.

Las decisiones documentadas en este capítulo no solo responden a los requerimientos funcionales establecidos para la versión 1.0, sino que también contemplan la evolución futura del proyecto mediante una arquitectura modular, escalable y preparada para incorporar nuevas funcionalidades sin comprometer la mantenibilidad de la aplicación.

El capítulo siguiente desarrolla la implementación técnica de esta arquitectura, describiendo las decisiones adoptadas durante el desarrollo del frontend, la organización del proyecto, el flujo de datos, la arquitectura implementada y los distintos componentes que conforman la versión final de Hartseer Analytics.

CAPITULO 2 - IMPLEMENTACIÓN DEL FRONTEND

INTRODUCCIÓN

Una vez definida la arquitectura conceptual presentada en el capítulo anterior, el siguiente paso consistió en trasladar dichas decisiones a una implementación funcional que permitiera materializar la totalidad de los principios establecidos durante la etapa de diseño.

El desarrollo del frontend de Hartseer Analytics fue abordado siguiendo una arquitectura modular, donde cada componente, módulo y capa del sistema fue implementado con una responsabilidad claramente definida. Este enfoque permitió mantener una separación consistente entre la presentación de la información, el procesamiento analítico y el acceso a los datos, favoreciendo tanto la mantenibilidad del proyecto como su capacidad de evolución.

Si bien la arquitectura conceptual planteaba originalmente una integración basada en una API REST como intermediaria entre la base de datos y la interfaz de usuario, durante el desarrollo de la versión 1.0 se adoptó una estrategia diferente. Con el objetivo de simplificar el proceso de despliegue, reducir la complejidad del proyecto y mantener el foco en el desarrollo analítico, se implementó una arquitectura intermedia basada en **SQL Views**, un proceso de exportación desarrollado en **Python** y archivos **JSON** utilizados como fuente de datos para la aplicación.

Esta decisión permitió conservar el principio de desacoplamiento definido durante la etapa de diseño, evitando que el frontend estableciera una comunicación directa con la base de datos y manteniendo una arquitectura preparada para incorporar una API REST en futuras versiones sin requerir modificaciones significativas sobre la lógica de presentación.

Como resultado, la implementación quedó organizada mediante un conjunto de módulos independientes, cada uno responsable de una función específica dentro del flujo de la aplicación. La carga de información, el procesamiento analítico, la gestión de los dashboards, el sistema de navegación y las visualizaciones fueron desarrollados como componentes desacoplados que colaboran entre sí para construir una aplicación consistente y fácilmente escalable.

Las siguientes secciones describen la organización del proyecto, la arquitectura finalmente implementada y el funcionamiento de cada una de las capas que

conforman el frontend de Hartseer Analytics, documentando las decisiones adoptadas durante el desarrollo y el modo en que dichas decisiones dieron forma a la versión final de la aplicación.

ARQUITECTURA IMPLEMENTADA

La implementación del frontend de Hartseer Analytics fue desarrollada siguiendo una arquitectura modular basada en la separación de responsabilidades, donde cada capa del sistema cumple una función específica dentro del flujo de procesamiento de la información.

A diferencia de la arquitectura conceptual presentada durante la etapa de diseño, la versión 1.0 incorporó una capa intermedia de procesamiento de datos que permitió desacoplar completamente la interfaz de usuario del modelo relacional implementado en MySQL. Esta decisión simplificó el despliegue de la aplicación, redujo la complejidad del proyecto y mantuvo una estructura preparada para futuras ampliaciones.

Como resultado, la arquitectura implementada quedó organizada mediante el siguiente flujo:

MySQL

↓

SQL Views

↓

Python Export

↓

JSON

↓

Data Service

↓

Analytics Engine

↓

Dashboard Controller

↓

Chart.js

↓

Frontend

Cada una de estas capas participa de manera independiente dentro del procesamiento de la información, colaborando entre sí mediante interfaces claramente definidas y evitando dependencias directas entre los distintos módulos de la aplicación.

La información comercial es almacenada en una base de datos MySQL y posteriormente consolidada mediante un conjunto de **SQL Views**, responsables de exponer estructuras de datos optimizadas para el análisis. Estas vistas constituyen la única fuente de información utilizada por el proceso de exportación desarrollado en Python.

El módulo de exportación consulta dichas vistas y genera archivos **JSON**, que actúan como fuente de datos para la aplicación durante la versión 1.0. De esta manera, el frontend evita establecer una conexión directa con la base de datos, preservando el desacoplamiento definido durante la etapa de diseño.

Una vez cargados los archivos JSON, el **Data Service** centraliza el acceso a la información y la pone a disposición del resto de los módulos de la aplicación. Sobre esta capa opera el **Analytics Engine**, encargado de transformar los datos en indicadores de negocio, comparaciones temporales, agregaciones y métricas reutilizables por todos los dashboards.

El **Dashboard Controller** coordina posteriormente la interacción entre la lógica analítica y la interfaz de usuario, gestionando la actualización de filtros, indicadores, gráficos y eventos sin incorporar reglas de negocio dentro de la capa de presentación.

Finalmente, **Chart.js** actúa como la biblioteca responsable de construir las visualizaciones analíticas, mientras que el frontend organiza y presenta toda la información mediante una arquitectura basada en Single Page Application (SPA), proporcionando una experiencia de usuario fluida y consistente.

La separación de responsabilidades implementada durante esta etapa permitió desarrollar un frontend altamente modular, donde cada componente puede evolucionar de manera independiente sin afectar el funcionamiento del resto de la

aplicación. Esta arquitectura facilita tanto el mantenimiento del proyecto como la incorporación de nuevas funcionalidades y constituye uno de los pilares fundamentales sobre los que se construyó Hartseer Analytics.

ORGANIZACIÓN DEL PROYECTO

Con el objetivo de mantener una arquitectura clara, modular y fácilmente mantenible, el frontend de Hartseer Analytics fue organizado mediante una estructura de directorios que separa cada uno de los recursos de la aplicación de acuerdo con su responsabilidad dentro del sistema.

Esta organización permite aislar la lógica de negocio, la presentación visual, los recursos estáticos y las herramientas auxiliares, favoreciendo tanto el mantenimiento del proyecto como la incorporación de nuevas funcionalidades sin alterar la estructura existente.

La organización general del proyecto se encuentra definida de la siguiente manera:

Hartseer Analytics

```
|
|
| └─ assets/
|
|   └─ icons/
|
|     └─ images/
|
|
| └─ css/
|
|   └─ components/
|
|     └─ layout/
|
|       └─ pages/
|
|         └─ styles.css
|
|
| └─ data/
|
|   └─ sales.json
|
|     └─ marketing-costs.json
```



```
|
|
|└─ js/
|
|  |└─ analytics.js
|
|  |└─ app.js
|
|  |└─ charts.js
|
|  |└─ dashboard-controller.js
|
|  |└─ data-service.js
|
|  └─ router.js
|
|
|└─ tools/
|
|  └─ export_to_json.py
|
|
|└─ views/
|
|
|└─ index.html
└─ README.md
```

Cada uno de estos directorios cumple una función específica dentro de la arquitectura de la aplicación.

La carpeta **assets** concentra los recursos gráficos utilizados por la interfaz, incluyendo iconografía e imágenes empleadas por los distintos dashboards y componentes visuales.

La carpeta **css** contiene la totalidad de los estilos de la aplicación, organizados según la responsabilidad de cada archivo. Esta separación permite mantener desacopladas las reglas correspondientes al layout general, los componentes reutilizables y las vistas específicas del sistema.

La carpeta **data** almacena los archivos JSON generados durante el proceso de exportación desde MySQL. Estos archivos constituyen la fuente de información consumida por el frontend durante la versión 1.0, evitando que la interfaz establezca una conexión directa con la base de datos.

La carpeta **js** reúne la totalidad de la lógica implementada en JavaScript. A diferencia de una organización basada únicamente en funcionalidades, los archivos fueron estructurados según la responsabilidad que cumplen dentro de la arquitectura del sistema. De esta manera, el acceso a los datos, el procesamiento analítico, el control de los dashboards, la navegación y las visualizaciones permanecen completamente desacoplados entre sí.

La carpeta **tools** incorpora las herramientas auxiliares utilizadas durante el desarrollo del proyecto. En la versión 1.0 contiene el script encargado de exportar la información desde las SQL Views hacia los archivos JSON consumidos posteriormente por la aplicación.

Por último, la carpeta **views** agrupa las distintas vistas que conforman la interfaz, mientras que el archivo **index.html** actúa como punto de entrada de la aplicación al implementar una arquitectura basada en Single Page Application (SPA).

Esta organización refleja uno de los principios fundamentales adoptados durante el desarrollo del frontend: cada módulo debe poseer una única responsabilidad claramente definida. Como resultado, la incorporación de nuevas funcionalidades puede realizarse mediante la extensión de módulos existentes o la incorporación de nuevos componentes, sin generar dependencias innecesarias entre las distintas capas de la aplicación.

CAPA DE DATOS

La capa de datos constituye el punto de partida del flujo de información implementado en el frontend de Hartseer Analytics. Su responsabilidad consiste en exponer una estructura de datos preparada para el análisis, desacoplando completamente la interfaz del modelo relacional definido en la base de datos.

Durante el desarrollo del proyecto se optó por incorporar una capa intermedia basada en **SQL Views**, evitando que la aplicación consumiera directamente las tablas transaccionales de MySQL. Esta decisión permitió centralizar la lógica de obtención de información, simplificar el acceso a los datos y reducir significativamente la complejidad del procesamiento realizado por el frontend.

La implementación de esta capa se basa en dos vistas principales:

- **Sales View**
- **Marketing Costs View**

La **Sales View** concentra la totalidad de la información comercial necesaria para construir los dashboards analíticos. Para ello consolida los datos provenientes de compras, clientes, productos, categorías, marcas y canales de venta, generando una estructura unificada que elimina la necesidad de realizar múltiples relaciones dentro de la aplicación.

Por su parte, la **Marketing Costs View** reúne la información correspondiente a la inversión realizada en cada canal de marketing, proporcionando los datos necesarios para el cálculo de indicadores como **Ad Spend, ROAS, Net Profit** y el resto de las métricas específicas del Dashboard de Marketing.

La utilización de SQL Views permitió trasladar la mayor parte de la lógica de integración de datos al motor de base de datos, aprovechando las capacidades de MySQL para consolidar la información antes de su consumo por parte del frontend. Como consecuencia, la interfaz recibe conjuntos de datos preparados para el análisis, reduciendo la cantidad de transformaciones necesarias durante la ejecución de la aplicación.

Durante el proceso de validación de esta capa se detectó una inconsistencia relacionada con la asociación entre productos y marcas. Determinados registros, correspondientes principalmente a productos cuya naturaleza no contemplaba una marca comercial asociada, eran descartados como consecuencia de una relación obligatoria implementada inicialmente mediante un **INNER JOIN**.

Para preservar la integridad del conjunto de datos se reemplazó dicha relación por un **LEFT JOIN**, permitiendo conservar la totalidad de las ventas registradas e identificar aquellos productos sin marca mediante un valor descriptivo. Esta corrección garantizó la consistencia de las métricas obtenidas posteriormente en Power BI y en el frontend de Hartseer Analytics.

Como resultado, la capa de datos proporciona una fuente de información consolidada, consistente y preparada para alimentar el resto de la arquitectura implementada, constituyendo el punto de partida sobre el cual se desarrolla el proceso de exportación y análisis descrito en las siguientes secciones.

PIPELINE DE EXPORTACIÓN MySQL → JSON

Una vez consolidada la información mediante las SQL Views, el siguiente paso dentro de la arquitectura implementada consiste en transformar dichos conjuntos de datos en un formato adecuado para ser consumido por el frontend.

Con este objetivo se desarrolló un proceso de exportación automatizado en **Python**, cuya responsabilidad consiste en consultar las vistas analíticas definidas en MySQL y generar los archivos **JSON** utilizados posteriormente por la aplicación.

El proceso de exportación forma parte de una capa intermedia de transformación de datos y permite desacoplar completamente la interfaz del motor de base de datos. De esta manera, el frontend consume información previamente procesada y estructurada para el análisis, evitando establecer conexiones directas con MySQL o ejecutar consultas SQL durante la interacción del usuario.

El flujo implementado durante la versión 1.0 puede representarse mediante el siguiente esquema:

MySQL

↓

SQL Views

↓

Python Export

↓

JSON

↓

Frontend

Durante cada proceso de exportación, el script desarrollado en Python establece una conexión con la base de datos, consulta las vistas analíticas implementadas para el proyecto y transforma los resultados obtenidos en archivos JSON independientes. Cada archivo representa un conjunto de información específico y mantiene una estructura consistente que facilita su posterior consumo por parte del frontend.

Esta estrategia ofrece diversas ventajas dentro de la arquitectura implementada. En primer lugar, elimina la necesidad de mantener un servidor de aplicaciones ejecutándose durante el desarrollo de la interfaz. Asimismo, reduce la complejidad del despliegue, simplifica el proceso de distribución del proyecto y permite trabajar sobre conjuntos de datos estáticos completamente validados antes de ser utilizados por la aplicación.

Desde el punto de vista del rendimiento, el frontend accede directamente a archivos JSON previamente generados, evitando tiempos de respuesta asociados a consultas sobre la base de datos durante la navegación entre dashboards. Como consecuencia, la carga de información resulta más rápida y completamente independiente del estado del servidor de base de datos.

Si bien esta arquitectura fue adoptada específicamente para la versión 1.0 de Hartseer Analytics, su implementación preserva la separación de responsabilidades definida desde las primeras etapas del proyecto. En futuras versiones, la capa de exportación podrá ser sustituida por una API REST sin modificar la lógica analítica ni la arquitectura general del frontend, ya que los módulos responsables del procesamiento de la información permanecen completamente desacoplados del mecanismo utilizado para obtener los datos.

En consecuencia, el proceso de exportación MySQL → JSON no constituye únicamente una herramienta de generación de archivos, sino una capa de transición que permite conectar el modelo relacional desarrollado para el proyecto con la arquitectura modular implementada en el frontend.

DATA SERVICE

Una vez generado el conjunto de archivos JSON mediante el proceso de exportación, el primer módulo responsable de interactuar con dicha información dentro del frontend es el **Data Service**.

Este componente constituye la capa de acceso a datos de la aplicación y tiene como responsabilidad centralizar la carga, almacenamiento y distribución de la información utilizada por los distintos módulos analíticos. De esta manera, el resto de la arquitectura permanece completamente desacoplado del origen de los datos, interactuando únicamente con una interfaz unificada para su consulta.

Durante la inicialización de la aplicación, el Data Service carga los archivos JSON generados por el proceso de exportación y los mantiene disponibles en memoria para el resto de los componentes del sistema. Como resultado, la información es

leída una única vez durante el ciclo de vida de la aplicación, evitando cargas repetitivas y reduciendo el tiempo de respuesta durante la navegación entre dashboards.

La responsabilidad de este módulo se limita exclusivamente a la gestión de los datos. En ningún momento realiza cálculos analíticos, transformaciones de negocio o modificaciones sobre la información obtenida desde la capa de datos. Su única función consiste en proporcionar un mecanismo uniforme para acceder a los distintos conjuntos de información consumidos por la aplicación.

Esta separación permite que los módulos encargados del procesamiento analítico permanezcan completamente independientes del mecanismo utilizado para obtener los datos. En consecuencia, el **Analytics Engine** puede concentrarse exclusivamente en el cálculo de indicadores de negocio, mientras que los controladores de cada dashboard únicamente consumen la información ya procesada para construir la interfaz de usuario.

Desde el punto de vista arquitectónico, el Data Service actúa como una capa de abstracción entre la persistencia de los datos y el resto del frontend. Gracias a esta decisión, una futura sustitución de los archivos JSON por una API REST únicamente requerirá modificar este módulo, preservando sin cambios el funcionamiento del Analytics Engine, los Dashboard Controllers y el resto de la aplicación.

Como resultado, el Data Service constituye uno de los principales mecanismos de desacoplamiento implementados durante el desarrollo del frontend, permitiendo mantener una arquitectura modular, reutilizable y preparada para evolucionar sin afectar la lógica analítica ni la capa de presentación.

ANALYTICS ENGINE

Una vez que la información es cargada por el **Data Service**, el siguiente componente dentro de la arquitectura del frontend es el **Analytics Engine**, responsable de transformar los datos obtenidos desde la capa de persistencia en indicadores analíticos reutilizables por toda la aplicación.

Este módulo concentra la totalidad de la lógica de negocio implementada en el frontend y constituye el núcleo analítico de Hartseer Analytics. Su responsabilidad consiste en procesar la información proveniente de los archivos JSON, realizar los cálculos necesarios y exponer métricas listas para ser consumidas por los

distintos dashboards sin incorporar lógica analítica dentro de la interfaz de usuario.

La separación entre acceso a datos y procesamiento analítico permite mantener una arquitectura claramente desacoplada. Mientras el **Data Service** se limita a proporcionar información, el Analytics Engine interpreta dichos datos y los transforma en indicadores de negocio, comparaciones temporales, agregaciones y estructuras preparadas para su representación gráfica.

Entre las principales responsabilidades implementadas por este módulo se encuentran:

- Cálculo de indicadores financieros y comerciales.
- Agregación de información por períodos temporales.
- Comparación automática entre períodos equivalentes.
- Construcción de series temporales para gráficos.
- Agrupación de información por categorías, productos, clientes y canales.
- Cálculo de porcentajes, participaciones y variaciones.
- Generación de estructuras reutilizables para los distintos dashboards.

Esta arquitectura evita que los controladores de cada dashboard deban implementar reglas de negocio propias. Como consecuencia, todos los indicadores utilizados por la aplicación son calculados desde un único punto, garantizando consistencia entre los distintos módulos y reduciendo significativamente la duplicación de código.

Durante el desarrollo del proyecto, el Analytics Engine fue evolucionando progresivamente hasta convertirse en un módulo completamente independiente de la interfaz de usuario. Inicialmente, diversos cálculos se encontraban distribuidos entre los dashboards; sin embargo, a medida que aumentó la complejidad de la aplicación, se decidió centralizar toda la lógica analítica en un único componente especializado.

Esta decisión permitió reutilizar funciones entre dashboards, simplificar el mantenimiento del código y asegurar que cualquier modificación realizada sobre una métrica de negocio impactara de forma uniforme en toda la aplicación.

Como resultado, el Analytics Engine implementa actualmente el cálculo de indicadores fundamentales para el análisis del negocio, entre ellos **Revenue**, **Profit**, **Margin**, **Average Order Value (AOV)**, **Return on Ad Spend (ROAS)**, cantidad de órdenes, cantidad de clientes, inversión en marketing y el resto de las métricas utilizadas por los distintos dashboards de Hartseer Analytics.

Además del cálculo de indicadores, este módulo incorpora funciones auxiliares destinadas al procesamiento de períodos de tiempo, normalización de fechas, comparaciones entre intervalos equivalentes y generación de estructuras optimizadas para la construcción de gráficos mediante Chart.js.

La centralización de la lógica analítica constituye uno de los principios arquitectónicos más importantes del frontend implementado. Gracias a esta decisión, la capa de presentación permanece completamente desacoplada del procesamiento de negocio, permitiendo que los dashboards se limiten exclusivamente a representar la información calculada por el Analytics Engine.

DASHBOARD CONTROLLER

Una vez procesada la información por el **Analytics Engine**, el siguiente componente dentro de la arquitectura del frontend es el **Dashboard Controller**, responsable de coordinar la interacción entre la lógica analítica y la interfaz de usuario.

Este módulo constituye la capa de orquestación de la aplicación y tiene como finalidad gestionar el comportamiento de cada dashboard sin incorporar reglas de negocio ni lógica de procesamiento analítico. Su responsabilidad consiste exclusivamente en coordinar la actualización de los distintos componentes visuales a partir de la información generada por el Analytics Engine.

Durante el desarrollo del proyecto se optó por implementar un controlador independiente con el objetivo de evitar que cada dashboard administrara de forma autónoma su propio flujo de ejecución. Esta decisión permitió centralizar la gestión de eventos, reducir la duplicación de código y establecer un comportamiento uniforme para todos los módulos analíticos de la aplicación.

El Dashboard Controller coordina las distintas acciones ejecutadas durante la interacción del usuario, entre ellas:

- Inicialización de cada dashboard.
- Gestión del ciclo de actualización de la interfaz.
- Coordinación de los filtros aplicados por el usuario.
- Solicitud de información al Analytics Engine.
- Actualización de indicadores (KPI Cards).
- Actualización de visualizaciones analíticas.
- Sincronización del estado general del dashboard.

De esta manera, los distintos componentes visuales permanecen completamente desacoplados de la lógica de negocio, limitándose a representar la información recibida sin conocer el proceso mediante el cual fue obtenida o calculada.

Esta arquitectura permite que un mismo conjunto de funciones analíticas pueda ser reutilizado por múltiples dashboards, manteniendo una única implementación para el procesamiento de datos y delegando al Dashboard Controller la responsabilidad de decidir cuándo y cómo debe actualizarse cada componente de la interfaz.

Asimismo, la incorporación de este módulo simplificó significativamente el mantenimiento del proyecto. La gestión de eventos, filtros y actualizaciones dejó de encontrarse distribuida entre múltiples archivos, pasando a centralizarse en una única capa de coordinación responsable del flujo general de la aplicación.

Como resultado, el Dashboard Controller actúa como el punto de conexión entre la lógica analítica y la capa de presentación, permitiendo mantener una arquitectura organizada, modular y fácilmente escalable sin comprometer la independencia de los distintos módulos que conforman el frontend de Hartseer Analytics.

SISTEMA DE FILTROS

El sistema de filtros constituye uno de los principales mecanismos de interacción implementados en el frontend de Hartseer Analytics. Su objetivo consiste en permitir que el usuario explore distintos períodos y dimensiones de análisis sin alterar la estructura general de la aplicación, actualizando dinámicamente la totalidad de los indicadores y visualizaciones correspondientes.

Desde el punto de vista arquitectónico, los filtros fueron concebidos como un componente transversal a toda la aplicación. En lugar de limitarse a modificar elementos individuales de la interfaz, cada interacción realizada por el usuario desencadena un proceso coordinado que involucra al Dashboard Controller, al Analytics Engine y a los distintos componentes de visualización.

Cuando un filtro es modificado, el Dashboard Controller captura el evento generado por la interfaz y solicita al Analytics Engine el recálculo de la información correspondiente al nuevo período o dimensión seleccionada. Una vez finalizado el procesamiento, los resultados son distribuidos a las KPI Cards y a las visualizaciones analíticas, garantizando que todos los componentes del

dashboard permanezcan sincronizados y representen exactamente el mismo conjunto de datos.

La implementación contempla distintos tipos de filtros según las necesidades analíticas de cada módulo:

- **Executive Dashboard:** filtro por rango de fechas.
- **Products Dashboard:** filtro por rango de fechas y selector de agrupación.
- **Customers Dashboard:** filtro por rango de fechas.
- **Marketing Dashboard:** filtro por rango de fechas y filtro por canal.

Esta distribución permite adaptar la interacción de cada dashboard a los objetivos específicos de análisis, manteniendo al mismo tiempo una experiencia de usuario uniforme en toda la aplicación.

Uno de los aspectos más relevantes del sistema implementado consiste en la comparación automática entre períodos equivalentes. Por defecto, los dashboards analíticos presentan la información correspondiente al último período disponible dentro del conjunto de datos y generan automáticamente la comparación con el período inmediatamente anterior de igual duración. Esta funcionalidad permite mostrar la evolución de los indicadores sin requerir configuraciones adicionales por parte del usuario.

Durante el desarrollo también se incorporaron distintos mecanismos de validación destinados a preservar la consistencia de la información analizada. Entre ellos se incluyen la normalización automática de fechas, la sincronización de los límites mínimo y máximo de los controles de selección, la validación de rangos inválidos y la corrección automática de intervalos incompatibles, evitando que el usuario pueda generar consultas inconsistentes dentro de la aplicación.

La arquitectura implementada permite que todas estas validaciones permanezcan centralizadas, garantizando un comportamiento homogéneo entre los distintos dashboards y reduciendo la necesidad de implementar controles específicos para cada módulo.

Como resultado, el sistema de filtros no solo actúa como un mecanismo de interacción entre el usuario y la aplicación, sino también como un componente responsable de preservar la coherencia analítica de toda la información

presentada por Hartseer Analytics, asegurando que indicadores, comparaciones y visualizaciones respondan siempre al mismo contexto de análisis.

SISTEMA DE NAVEGACIÓN SPA

Uno de los principios arquitectónicos adoptados durante el desarrollo del frontend de Hartseer Analytics consistió en implementar un sistema de navegación basado en el modelo **Single Page Application (SPA)**. Esta arquitectura permite que la totalidad de la aplicación funcione sobre un único documento HTML, actualizando únicamente el contenido necesario durante la navegación entre los distintos dashboards.

A diferencia de una aplicación web tradicional, donde cada cambio de sección implica la carga de una nueva página, el enfoque SPA mantiene activos los componentes compartidos de la interfaz y reemplaza únicamente el contenido principal correspondiente al módulo seleccionado. Como consecuencia, la navegación resulta más fluida, reduce los tiempos de carga y proporciona una experiencia de usuario más consistente.

La implementación desarrollada para Hartseer Analytics se basa en un sistema de **Hash Routing**, donde cada dashboard se identifica mediante un fragmento específico dentro de la URL. Esta estrategia permite administrar la navegación sin requerir un servidor de aplicaciones ni configuraciones adicionales, manteniendo la simplicidad de despliegue establecida para la versión 1.0.

Durante la interacción del usuario, cada selección realizada sobre la Sidebar modifica el estado de la navegación y actualiza dinámicamente el contenido principal de la aplicación. Este proceso es gestionado por el **Router**, responsable de identificar el dashboard solicitado, cargar la vista correspondiente e iniciar el flujo de actualización necesario para representar la información analítica.

La arquitectura implementada mantiene permanentemente visibles los componentes compartidos de la aplicación, entre ellos la Sidebar, la estructura principal del layout y los estilos globales. De esta manera, únicamente los elementos específicos de cada dashboard son reemplazados durante la navegación, reduciendo la cantidad de operaciones realizadas por el navegador y mejorando la continuidad de la experiencia de usuario.

Esta decisión permitió reutilizar la totalidad de los componentes comunes entre dashboards, simplificar la organización del código y mantener una arquitectura fácilmente escalable para futuras incorporaciones de nuevos módulos analíticos.

Asimismo, la utilización de Hash Routing proporciona una separación clara entre el sistema de navegación y la lógica de negocio implementada por la aplicación. El Router se limita exclusivamente a administrar el cambio de vistas, mientras que el Dashboard Controller y el Analytics Engine permanecen responsables de la actualización de la información y del procesamiento analítico respectivamente.

Como resultado, el sistema de navegación SPA implementado en Hartseer Analytics constituye uno de los pilares fundamentales de la arquitectura del frontend, permitiendo combinar simplicidad de implementación, reutilización de componentes y una experiencia de usuario comparable a la ofrecida por aplicaciones modernas de Business Intelligence.

IMPLEMENTACIÓN DEL DASHBOARD

La interfaz principal de Hartseer Analytics fue implementada mediante un conjunto de dashboards especializados, cada uno orientado al análisis de un área específica del negocio. Si bien cada módulo presenta indicadores y visualizaciones particulares, todos fueron desarrollados siguiendo una misma estructura arquitectónica, garantizando consistencia visual, reutilización de componentes y uniformidad en la interacción del usuario.

Los dashboards implementados en la versión 1.0 son los siguientes:

- Executive Dashboard
- Products Dashboard
- Customers Dashboard
- Marketing Dashboard

Cada uno de ellos representa una vista independiente dentro de la aplicación, aunque todos comparten la misma arquitectura de funcionamiento. Esta decisión permitió desarrollar una interfaz homogénea, donde la experiencia de navegación permanece constante independientemente del módulo analítico seleccionado.

La implementación de cada dashboard se encuentra organizada mediante cuatro bloques funcionales claramente diferenciados:

- Encabezado (Dashboard Header)

- Sistema de Filtros
- Indicadores (KPI Cards)
- Visualizaciones Analíticas

El **Dashboard Header** proporciona el contexto general de la vista, incorporando el título del módulo, una breve descripción y los controles de filtrado correspondientes. Este componente permanece consistente entre todos los dashboards, variando únicamente los filtros específicos requeridos por cada área de análisis.

A continuación se ubican las **KPI Cards**, responsables de presentar los principales indicadores de negocio asociados al período seleccionado. Cada tarjeta muestra el valor actual de la métrica, su variación respecto del período de comparación y una representación visual del comportamiento del indicador, proporcionando una visión ejecutiva inmediata del estado del negocio.

Finalmente, cada dashboard incorpora un conjunto de visualizaciones analíticas implementadas mediante **Chart.js**, organizadas de acuerdo con la naturaleza de la información representada. La selección del tipo de gráfico responde a criterios definidos durante la etapa de diseño analítico, buscando facilitar la interpretación de tendencias, distribuciones y comparaciones.

Si bien todos los dashboards comparten la misma estructura base, cada uno incorpora funcionalidades específicas adaptadas al dominio de análisis correspondiente. El Dashboard de Productos permite modificar la dimensión de agrupación utilizada para el análisis, mientras que el Dashboard de Marketing incorpora filtros específicos por canal. Por su parte, los dashboards Ejecutivo y de Clientes priorizan el análisis temporal mediante filtros basados exclusivamente en rangos de fechas.

Esta estrategia permitió construir una arquitectura altamente reutilizable, donde la mayor parte de los componentes visuales son compartidos entre módulos, reduciendo la duplicación de código y facilitando la incorporación de nuevos dashboards sin modificar la estructura general de la aplicación.

Como resultado, la implementación de los dashboards mantiene un equilibrio entre consistencia visual y especialización analítica, permitiendo que cada módulo responda a necesidades específicas del negocio sin perder coherencia con el resto del sistema.

SISTEMA DE VISUALIZACIÓN

La última capa de la arquitectura implementada corresponde al sistema de visualización, responsable de transformar los indicadores calculados por el Analytics Engine en representaciones gráficas que faciliten la interpretación de la información por parte del usuario.

Durante el desarrollo del proyecto se optó por utilizar **Chart.js** como biblioteca para la construcción de las visualizaciones analíticas, debido a su simplicidad de integración, bajo nivel de dependencias y capacidad para generar gráficos interactivos manteniendo una implementación ligera y fácilmente mantenible.

La arquitectura implementada desacopla completamente la generación de datos de su representación visual. Mientras que el Analytics Engine produce estructuras de información preparadas para el análisis, el sistema de visualización se limita exclusivamente a representar dichos resultados mediante gráficos e indicadores, evitando incorporar cálculos o reglas de negocio dentro de la capa gráfica.

Para mantener una experiencia uniforme entre los distintos dashboards, se desarrolló una capa de configuración compartida que centraliza la definición de estilos, colores, formatos numéricos y opciones comunes utilizadas por todas las visualizaciones. Esta decisión permitió garantizar consistencia visual entre los distintos módulos y reducir significativamente la duplicación de código.

La selección del tipo de gráfico utilizado en cada dashboard responde a los criterios establecidos durante la etapa de diseño analítico. En consecuencia, cada visualización fue elegida considerando la naturaleza de la información representada y el objetivo específico del análisis, priorizando siempre la claridad por encima de la complejidad visual.

Entre las principales visualizaciones implementadas se incluyen:

- Gráficos de líneas para representar tendencias temporales.
- Gráficos de barras para comparar magnitudes entre categorías.
- Gráficos de dona para visualizar distribuciones porcentuales.
- Indicadores KPI para resumir el estado general del negocio.

Cada una de estas visualizaciones responde dinámicamente a las acciones realizadas por el usuario sobre el sistema de filtros. Ante cualquier modificación del período de análisis o de la dimensión seleccionada, el Dashboard Controller coordina el recálculo de la información y actualiza automáticamente todos los componentes gráficos, garantizando que la totalidad del dashboard permanezca sincronizada con el contexto analítico seleccionado.

Asimismo, durante la implementación se incorporó un mecanismo de gestión del ciclo de vida de los gráficos, destruyendo las instancias previamente renderizadas antes de generar nuevas visualizaciones. Esta estrategia evita la superposición de gráficos, reduce el consumo innecesario de memoria y asegura un comportamiento consistente durante las actualizaciones sucesivas de cada dashboard.

Como resultado, el sistema de visualización implementado constituye la capa encargada de materializar toda la información procesada por la arquitectura analítica, permitiendo presentar indicadores y gráficos de forma clara, consistente e interactiva, manteniendo una experiencia de usuario alineada con los principios definidos durante la etapa de diseño del frontend.

CONCLUSIÓN

La implementación del frontend de Hartseer Analytics permitió materializar la arquitectura conceptual definida durante la etapa de diseño, consolidando una aplicación web capaz de integrar la totalidad del proceso analítico desarrollado a lo largo del proyecto.

La organización modular adoptada durante el desarrollo permitió separar claramente las responsabilidades de acceso a datos, procesamiento analítico, navegación, control de la interfaz y visualización de la información. Como resultado, cada componente de la aplicación cumple una función específica dentro de la arquitectura general, favoreciendo el mantenimiento del código, la reutilización de módulos y la incorporación de futuras funcionalidades sin comprometer la estabilidad del sistema.

La implementación de una capa de datos basada en **SQL Views**, un proceso de exportación mediante **Python**, una arquitectura de acceso centralizada a través del **Data Service**, un **Analytics Engine** independiente de la interfaz y un sistema de navegación **Single Page Application (SPA)** permitió construir una solución

desacoplada y consistente, donde cada módulo colabora con el resto mediante interfaces claramente definidas.

Asimismo, el desarrollo del frontend permitió trasladar los indicadores previamente implementados en Power BI hacia una aplicación web interactiva, manteniendo la consistencia de las métricas y proporcionando una experiencia de usuario más cercana a la utilizada por plataformas profesionales de Business Intelligence.

Desde una perspectiva de ingeniería de software, la implementación realizada demuestra la importancia de una arquitectura basada en la separación de responsabilidades. La independencia entre las distintas capas del sistema facilita tanto el mantenimiento como la evolución futura del proyecto, permitiendo sustituir componentes individuales —como la actual capa de archivos JSON por una API REST— sin modificar la lógica analítica ni la interfaz de usuario.

En conjunto, la implementación desarrollada para Hartseer Analytics constituye una solución sólida, modular y preparada para evolucionar, representando la etapa final de un proyecto que integra modelado de datos, consultas SQL, análisis de negocio, visualización analítica y desarrollo frontend dentro de una única aplicación orientada al análisis de información.

CAPITULO 3 - DECISIONES TÉCNICAS Y VALIDACIÓN DEL PROYECTO

DECISIONES TECNICAS

El desarrollo del frontend de Hartseer Analytics implicó la adopción de diversas decisiones técnicas destinadas a equilibrar simplicidad de implementación, mantenibilidad y capacidad de evolución del proyecto. Cada una de estas decisiones fue evaluada considerando el alcance definido para la versión 1.0 y la necesidad de construir una arquitectura sólida que pudiera evolucionar progresivamente sin requerir una reestructuración completa de la aplicación.

Una de las decisiones más importantes consistió en desarrollar el frontend utilizando exclusivamente **HTML5, CSS3 y JavaScript Vanilla**, descartando la utilización de frameworks como React, Angular o Vue. Esta elección permitió fortalecer los fundamentos del desarrollo frontend, reducir la complejidad del proyecto y mantener un control completo sobre cada una de las capas implementadas. Asimismo, evitó incorporar dependencias externas que excedieran los objetivos definidos para esta etapa.

Otra decisión relevante fue la adopción de una arquitectura **Single Page Application (SPA)** implementada mediante un sistema de **Hash Routing**. Esta estrategia permitió construir una experiencia de navegación fluida sin depender de un servidor de aplicaciones ni de configuraciones específicas para el manejo de rutas, simplificando tanto el desarrollo como el proceso de despliegue del proyecto.

Durante la etapa de implementación también se redefinió el mecanismo de acceso a los datos. Si bien la arquitectura conceptual contemplaba inicialmente el consumo de información mediante una API REST, se optó por implementar una

solución intermedia basada en **SQL Views**, un proceso de exportación desarrollado en Python y archivos JSON utilizados como fuente de datos por el frontend. Esta decisión permitió desacoplar completamente la interfaz del modelo relacional, reducir la complejidad de la infraestructura necesaria para ejecutar la aplicación y mantener una arquitectura preparada para incorporar una API REST en futuras versiones.

Desde el punto de vista de la organización interna del proyecto, se decidió centralizar la lógica de acceso a los datos, el procesamiento analítico y la coordinación de la interfaz en módulos independientes. Como resultado surgieron componentes especializados como el **Data Service**, el **Analytics Engine** y el **Dashboard Controller**, cada uno responsable de una única función dentro del flujo general de la aplicación. Esta separación permitió reducir el acoplamiento entre módulos, mejorar la reutilización de código y simplificar el mantenimiento del sistema.

Otra decisión significativa consistió en desarrollar un sistema de componentes reutilizables para la construcción de los distintos dashboards. Tanto las KPI Cards como los filtros, encabezados y tarjetas de gráficos fueron implementados siguiendo una estructura común, garantizando consistencia visual y reduciendo la duplicación de código entre los distintos módulos analíticos.

Finalmente, se priorizó una filosofía de desarrollo basada en la simplicidad. Cada funcionalidad incorporada durante la versión 1.0 respondió a una necesidad claramente identificada dentro del alcance del proyecto, evitando introducir mecanismos innecesariamente complejos que incrementaran la dificultad de mantenimiento o desviaran el objetivo principal de la aplicación.

En conjunto, estas decisiones técnicas permitieron construir un frontend modular, desacoplado y fácilmente escalable, donde cada componente cumple una responsabilidad claramente definida y la arquitectura permanece preparada para evolucionar de manera progresiva conforme se incorporen nuevas funcionalidades en versiones futuras.

VALIDACION Y ASEGURAMIENTO DE CALIDA (QA)

Una vez finalizada la implementación del frontend, se llevó a cabo un proceso integral de validación técnica cuyo objetivo consistió en verificar la consistencia funcional, analítica y arquitectónica de la aplicación antes de considerar concluida la versión 1.0 del proyecto.

El proceso de aseguramiento de la calidad (Quality Assurance – QA) fue desarrollado de manera incremental, evaluando cada uno de los componentes implementados conforme avanzaba el desarrollo. Esta metodología permitió detectar inconsistencias en etapas tempranas, simplificando su corrección y evitando la propagación de errores hacia el resto de la arquitectura.

La validación comenzó con la comprobación de la integridad de los datos obtenidos desde la capa de persistencia. Para ello se contrastaron los resultados producidos por las SQL Views con la información representada en los dashboards desarrollados previamente en Power BI, verificando que la totalidad de los indicadores financieros y comerciales coincidieran entre ambas implementaciones.

Posteriormente se validó el funcionamiento del **Analytics Engine**, comprobando el cálculo correcto de indicadores, comparaciones temporales, agregaciones y métricas derivadas utilizadas por los distintos dashboards. Este proceso permitió confirmar que la lógica analítica implementada en el frontend reproducía fielmente los resultados obtenidos durante la etapa de análisis de datos.

El sistema de filtros también fue sometido a un proceso específico de validación. Se verificó el correcto funcionamiento de los rangos de fechas, la sincronización automática entre los controles **From** y **To**, la normalización de intervalos inválidos, la comparación automática con períodos equivalentes y la actualización consistente de KPI Cards y visualizaciones frente a cada interacción realizada por el usuario.

Del mismo modo, se evaluó el comportamiento del sistema de navegación SPA, comprobando el funcionamiento del Router, la persistencia de la interfaz, la carga dinámica de los dashboards y la correcta inicialización de cada módulo durante la navegación entre las distintas vistas de la aplicación.

En relación con el sistema de visualización, se verificó la correcta actualización de los gráficos implementados mediante Chart.js, validando especialmente la destrucción de instancias previamente renderizadas antes de generar nuevas visualizaciones. Esta comprobación permitió evitar superposiciones gráficas, pérdidas de memoria y comportamientos inconsistentes durante el cambio de filtros o de dashboard.

Como parte del proceso de revisión final también se realizaron tareas de aseguramiento de la calidad orientadas a la arquitectura general del proyecto. Entre ellas se incluyeron la revisión de la estructura de directorios, la eliminación de archivos innecesarios para producción, la configuración del archivo **.gitignore**, la limpieza de la consola del navegador, la verificación de dependencias entre

módulos y la comprobación del correcto funcionamiento del flujo completo de datos desde la capa de persistencia hasta la representación visual.

Finalmente, el proceso de validación concluyó con una revisión funcional completa de la aplicación, recorriendo la totalidad de los dashboards implementados y verificando el comportamiento coordinado entre navegación, filtros, indicadores y visualizaciones. Esta última etapa permitió confirmar que todos los módulos respondían de manera consistente a la arquitectura definida durante el diseño y que la versión 1.0 cumplía con los objetivos funcionales establecidos para el proyecto.

Como resultado, el proceso de Quality Assurance permitió garantizar la consistencia analítica, funcional y arquitectónica del frontend de Hartseer Analytics, proporcionando una base estable sobre la cual podrán desarrollarse futuras versiones del sistema.

ROADMAP TECNICO

Si bien la arquitectura implementada satisface la totalidad de los objetivos definidos para la versión 1.0, el proyecto fue desarrollado contemplando su evolución técnica a mediano y largo plazo. Desde las primeras etapas del diseño se priorizó una estructura modular y desacoplada con el propósito de facilitar la incorporación de nuevas tecnologías y funcionalidades sin requerir una reestructuración significativa de la aplicación.

Las principales líneas de evolución técnica previstas para futuras versiones se describen a continuación.

Sustitución de la capa JSON por una API REST

La arquitectura implementada incorpora actualmente una capa de datos basada en archivos JSON generados mediante un proceso de exportación desde MySQL. Si bien esta solución resultó adecuada para la versión inicial del proyecto, la evolución natural del sistema contempla reemplazar este mecanismo por una API REST capaz de proporcionar información en tiempo real.

Gracias al desacoplamiento implementado mediante el Data Service, esta transición podrá realizarse modificando únicamente la capa de acceso a datos, preservando sin cambios el funcionamiento del Analytics Engine, el Dashboard Controller y el resto de la arquitectura del frontend.

Integración con una base de datos en producción

La versión actual trabaja sobre un conjunto de datos previamente exportados y validados. Como evolución futura se prevé incorporar una base de datos conectada permanentemente a la aplicación, permitiendo reflejar cambios operativos en tiempo real y eliminando la necesidad de realizar procesos manuales de exportación.

Optimización del procesamiento analítico

A medida que aumente el volumen de información disponible, parte del procesamiento actualmente realizado por el Analytics Engine podrá trasladarse progresivamente hacia la capa de persistencia mediante consultas SQL optimizadas o procedimientos especializados, reduciendo la carga de procesamiento ejecutada por el navegador y mejorando el rendimiento general de la aplicación.

Incorporación de autenticación y control del acceso

La arquitectura implementada fue diseñada considerando la futura incorporación de mecanismos de autenticación y autorización de usuarios. Esta evolución permitirá restringir el acceso a los distintos módulos del sistema mediante perfiles de usuario y niveles de permisos diferenciados, acercando la aplicación a un entorno empresarial real.

Optimización para dispositivos móviles

Si bien la versión 1.0 fue desarrollada priorizando la experiencia de uso en equipos de escritorio, la arquitectura visual contempla la futura incorporación de un sistema de diseño responsive que permita adaptar la interfaz a distintos tamaños de pantalla sin modificar la organización general de los componentes.

Evolución de la arquitectura modular

La separación de responsabilidades implementada durante el desarrollo facilita la incorporación de nuevos dashboards, componentes analíticos y módulos especializados sin afectar el funcionamiento de la aplicación existente. Esta capacidad constituye uno de los principales objetivos perseguidos durante el diseño de la arquitectura y representa la base sobre la cual continuará evolucionando Hartseer Analytics.

En conjunto, estas líneas de evolución reflejan que la arquitectura desarrollada para la versión 1.0 no constituye una solución cerrada, sino el punto de partida de

una plataforma preparada para crecer de forma progresiva. La modularidad, el desacoplamiento entre componentes y la separación de responsabilidades implementadas durante el proyecto permiten incorporar nuevas capacidades preservando la estabilidad del sistema y manteniendo la coherencia arquitectónica alcanzada durante esta primera versión.

CONCLUSIÓN

El desarrollo de Hartseer Analytics permitió integrar en un único proyecto las distintas etapas que conforman un proceso completo de análisis de datos, abarcando desde el modelado de la base de datos y la construcción de consultas SQL hasta el desarrollo de dashboards analíticos en Power BI y la implementación de una aplicación web capaz de presentar dicha información de manera interactiva.

A lo largo del proyecto se priorizó la construcción de una solución organizada, modular y progresivamente escalable, donde cada etapa se desarrolló sobre los resultados obtenidos en la anterior. Esta metodología permitió mantener una visión integral del ciclo de vida de la información, evitando abordar cada tecnología como un elemento aislado y favoreciendo la comprensión de cómo interactúan las distintas capas que conforman una solución analítica real.

Desde el punto de vista técnico, la arquitectura implementada demostró la importancia de la separación de responsabilidades como principio fundamental de diseño. La incorporación de componentes especializados para el acceso a datos, el procesamiento analítico, la coordinación de la interfaz y la representación visual permitió construir una aplicación desacoplada, fácilmente mantenible y preparada para evolucionar sin comprometer la estabilidad del sistema.

Asimismo, el desarrollo del frontend representó la oportunidad de trasladar los indicadores previamente contruidos en Power BI hacia una aplicación propia, fortaleciendo la comprensión de conceptos relacionados con arquitectura de software, organización modular del código, reutilización de componentes y diseño de interfaces orientadas al análisis de información.

Si bien la versión 1.0 fue desarrollada con un alcance claramente definido, la arquitectura implementada fue concebida desde sus primeras etapas considerando la evolución futura del proyecto. La incorporación de nuevas funcionalidades, la migración hacia una API REST, la integración con una base de

datos en producción y el crecimiento del sistema podrán realizarse preservando los principios arquitectónicos establecidos durante esta primera versión.

Más allá del resultado técnico obtenido, Hartseer Analytics representa un ejercicio de integración de conocimientos provenientes de distintas disciplinas del área de datos y desarrollo de software. La construcción del proyecto requirió combinar criterios de modelado de datos, análisis de negocio, consultas SQL, visualización de información, arquitectura frontend y documentación técnica, demostrando la importancia de comprender cada una de estas etapas como parte de un único proceso orientado a transformar datos en información útil para la toma de decisiones.

Finalmente, este proyecto constituye el punto de partida para futuras evoluciones tanto desde el punto de vista funcional como arquitectónico. La experiencia obtenida durante su desarrollo permitió consolidar conocimientos técnicos, fortalecer criterios de diseño y comprender la relevancia de planificar, documentar y validar cada decisión antes de su implementación. En este sentido, Hartseer Analytics no representa únicamente el resultado de un proceso de desarrollo, sino también una base sólida sobre la cual continuar construyendo soluciones analíticas cada vez más completas y alineadas con las necesidades de un entorno profesional.

